

AFRILANG Stdlib

std/m/physics2

Bibliothèque AFRILANG `std/m/physics2` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : kineticEnergy, pot

```
`import "std/m/physics2" · `use physics2`
```

- `kineticEnergy(mass number, velocity number) ? number`
- `potentialEnergy(mass number, height number) ? number`
- `momentum(mass number, velocity number) ? number`
- `force(mass number, accel number) ? number`

Category: medium | Tier: medium | Functions: 4

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/m/physics2` (niveau medium, catégorie medium). Elle expose 4 fonction(s) : kineticEnergy, pot

```
`import "std/m/physics2"` · `use physics2`  
- `kineticEnergy(mass number, velocity number) ? number`  
- `potentialEnergy(mass number, height number) ? number`  
- `momentum(mass number, velocity number) ? number`  
- `force(mass number, accel number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/m/physics2"  
use physics2
```

Niveau : medium · Catégorie : Modules medium (m/)

Bibliothèques intermédiaires ? import std/m/?

3. Référence API

```
? kineticEnergy(mass number, velocity number) ? number  
? potentialEnergy(mass number, height number) ? number  
? momentum(mass number, velocity number) ? number  
? force(mass number, accel number) ? number
```

4. Exemples d'utilisation

```
import "std/m/physics2"  
use physics2  
  
create result = kineticEnergy(/* args */)   
say result  
  
create result = potentialEnergy(/* args */)   
say result  
  
create result = momentum(/* args */)   
say result  
  
create result = force(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/m/physics2"` en tête de fichier.  
? Lancez `afrilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module physics2  
  
function kineticEnergy(mass number, velocity number) returns number  
end  
  
function potentialEnergy(mass number, height number) returns number  
end  
  
function momentum(mass number, velocity number) returns number  
end  
  
function force(mass number, accel number) returns number  
end  
end
```

ENGLISH

1. Overview

AFRILANG library `std/m/physics2` (tier medium, category medium). Exports 4 function(s): kineticEnergy, potentialEnergy,

```
`import "std/m/physics2"` · `use physics2`  
- `kineticEnergy(mass number, velocity number) ? number`  
- `potentialEnergy(mass number, height number) ? number`  
- `momentum(mass number, velocity number) ? number`  
- `force(mass number, accel number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/m/physics2"  
use physics2
```

Tier: medium · Category: Medium modules (m/)
Intermediate libraries ? import std/m/?

3. API reference

```
? kineticEnergy(mass number, velocity number) ? number  
? potentialEnergy(mass number, height number) ? number  
? momentum(mass number, velocity number) ? number  
? force(mass number, accel number) ? number
```

4. Usage examples

```
import "std/m/physics2"  
use physics2  
  
create result = kineticEnergy(/* args */)   
say result  
  
create result = potentialEnergy(/* args */)   
say result  
  
create result = momentum(/* args */)   
say result  
  
create result = force(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/m/physics2"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module physics2  
  
function kineticEnergy(mass number, velocity number) returns number  
end  
  
function potentialEnergy(mass number, height number) returns number  
end  
  
function momentum(mass number, velocity number) returns number  
end  
  
function force(mass number, accel number) returns number  
end  
end
```


